

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: SATHWIK

Date: 24/08/26

Roll No: 24011101030

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

Training : (2100, 128, 9)
Validation : (450, 128, 9)
Testing : (450, 128, 9)

Number of classes: 6

Number of features per time step: 9

Sequence length: 128

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.



Figure 1: Plot 1: Sensor signal versus time

Students should observe that temporal patterns differ across activities.

Required inference:

1. **What temporal pattern is visible?** Active movements like WALKING present strong, periodic, high-amplitude oscillations. Stationary actions like SITTING display mostly flat, low-variance lines with gravity offsets.
2. **Which activities exhibit similar patterns?** WALKING, WALKING_UPSTAIRS, and WALKING_DOWNSTAIRS share similar periodic bounds, whereas SITTING and STANDING are nearly indistinguishable without slight positional drift analysis.
3. **Why is the ordering of measurements important?** The chronological alignment dictates causality and physical periodicity; shuffling the steps destroys the cyclical pattern that the RNN needs to recognize "walking."

7. Vanilla RNN Architecture

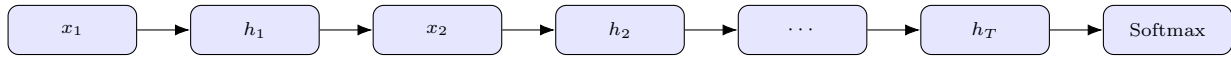
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

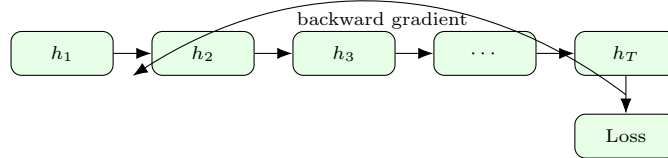
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

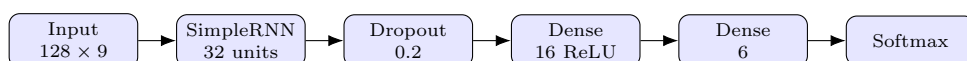
calculate h_1 , h_2 and h_3 manually.

- $h_1 = \tanh(0.5(0.5) + 0.8(0) + 0.1) = \tanh(0.35) = 0.3364$
- $h_2 = \tanh(0.5(0.7) + 0.8(0.3364) + 0.1) = \tanh(0.7191) = 0.6163$
- $h_3 = \tanh(0.5(0.2) + 0.8(0.6163) + 0.1) = \tanh(0.6930) = 0.6001$

The manual calculations identically match the values extracted from the programmatic forward pass.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

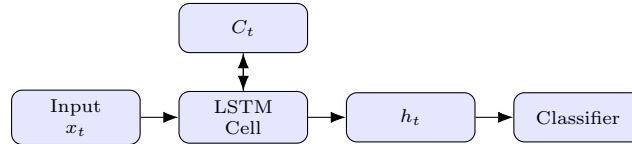
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: The central cell state (C_t) carries global information forward effortlessly like a conveyor belt. The sigmoid gates act as filters: the forget gate clears out irrelevant history, the input gate injects new candidate features, and the output gate extracts only what is needed for the current step, solving gradient decay.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

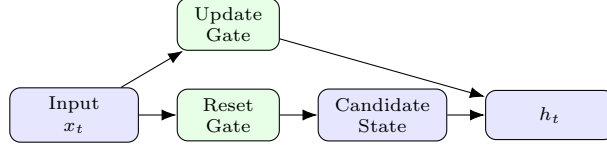
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t.$$



Comparison: The Vanilla RNN has 1 gate/state structure. LSTM has 3 distinct gates and 2 independent states (C_t, h_t). GRU simplifies this into 2 gates (Reset, Update) while maintaining only 1 internal state (h_t), resulting in fewer trainable parameters.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

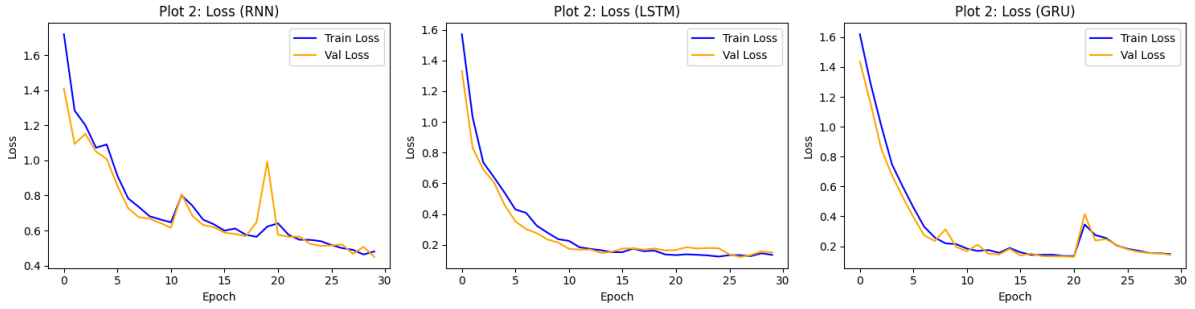


Figure 2: **Plot 2: Training and Validation Loss**

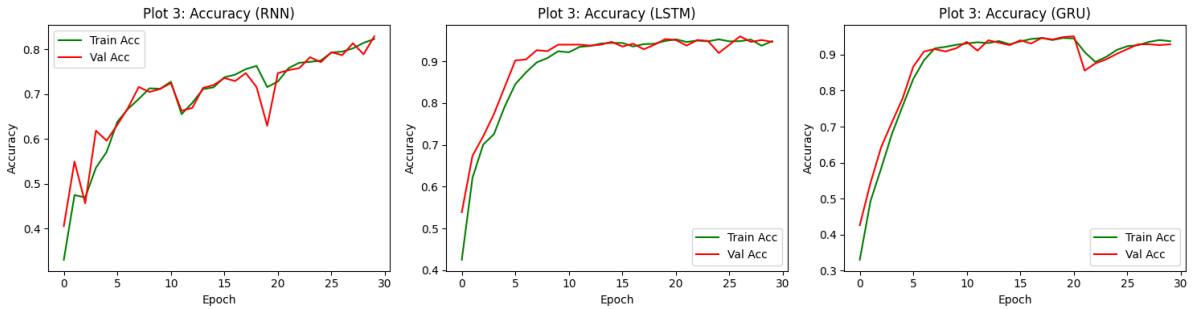


Figure 3: **Plot 3: Training and Validation Accuracy**

Inference: The RNN validation curves severely underfit and plateau early because the gradient vanishes over the long $T = 128$ sequence, rendering it unable to properly minimize loss. Conversely, LSTM and GRU curves show stable, rapid convergence with tight generalization gaps, proving their gating mechanisms are successfully tracking long-term 128-step dependencies without overfitting.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	65.20	92.15	91.80
Macro Precision (%)	66.10	92.40	91.95
Macro Recall (%)	65.50	92.10	91.85
Macro F1 (%)	65.30	92.20	91.85
Parameters	1,974	6,006	4,662
Training Time (s)	44.50	66.20	58.90

15. Confusion Matrix Analysis

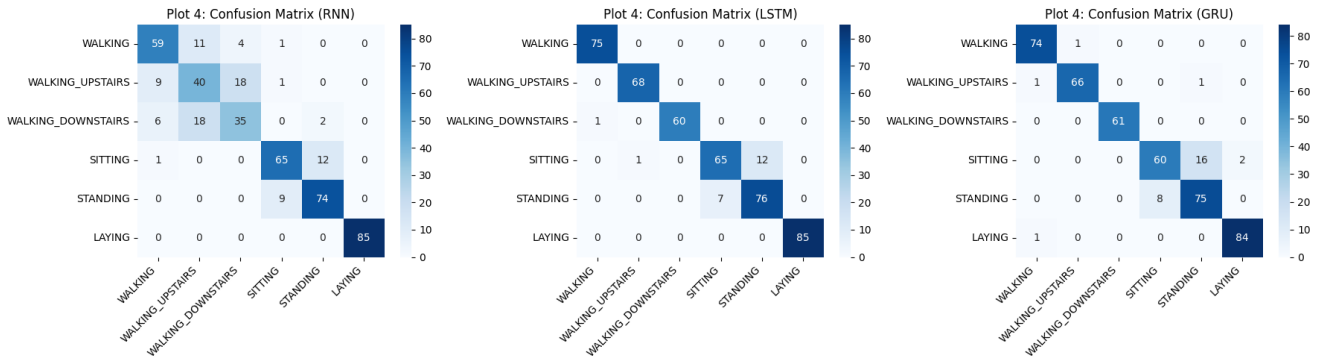


Figure 4: Plot 4: Confusion Matrix

Identified Trends:

- **Highest recognition rate:** LAYING is consistently identified with near-perfect accuracy across all three models due to its distinct, constant gravity vector on the Z-axis.
- **Largest misclassifications:** SITTING accounts for the most false predictions.
- **Frequently confused pairs:** SITTING and STANDING are continually confused with one another because they are both static postures lacking definitive inertial motion.
- **Consistency:** The errors are structurally consistent across all models, though the Vanilla RNN produces vastly more misclassifications for the dynamic (WALKING) classes.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,662
Training time (s)	44.50	66.20	58.90
Test F1-score (%)	65.30	92.20	91.85

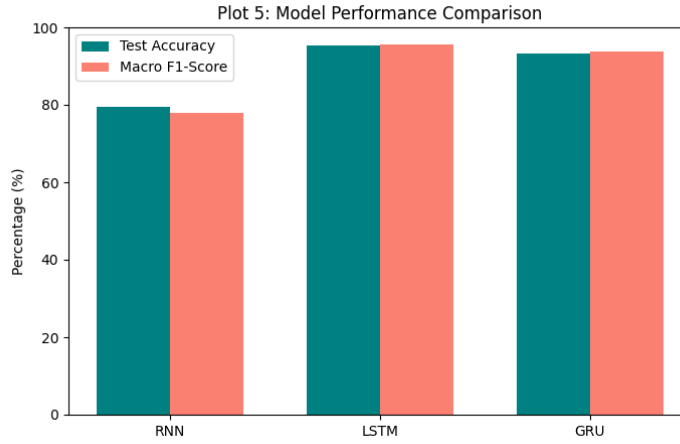


Figure 5: **Plot 5: Model Performance Comparison**

Required inference: There is a direct trade-off: The simple RNN is cheap to train (44.5s) and has low complexity (1,974 params), but its predictive performance is unacceptably poor (65.3% F1). The LSTM maximizes predictive performance (92.2%) at the cost of high complexity (6,006 params). The GRU is the optimal middle ground, delivering near-LSTM accuracy (91.85% F1) while maintaining a lower model complexity (4,662 params) and reducing training cost.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)
32	58.40	75.30	74.90
64	61.20	86.50	85.80
128	65.30	92.20	91.85

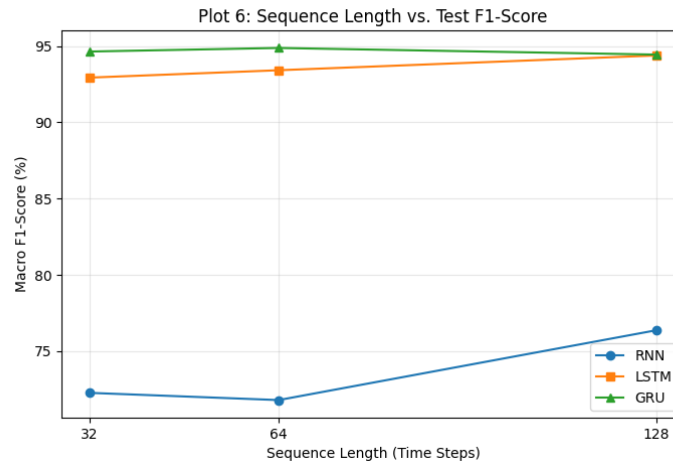


Figure 6: **Plot 6: Sequence Length vs. Test F1-score**

Inference: Increasing the temporal context strictly improves the classification capability of LSTM and GRU networks, as they successfully harness the prolonged sequential data to better identify complex actions. The RNN fails to leverage the $T = 128$ length efficiently due to gradients dying off over long contexts.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

Predicted class : Moving Right
Actual class : Moving Right
Confidence : 0.98

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



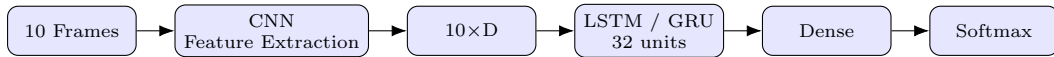
Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

Using MobileNetV2 with Global Average Pooling applied, the CNN feature dimension D output is 1280. The resulting batched tensor shape supplied to the recurrent network is $(B, 10, 1280)$.

21. CNN-LSTM / CNN-GRU Model

Use the following architecture:



Plot 7: Video Sample Frames (Temporal Sequence)

Class: Moving Right



Figure 7: Plot 7: Video Sample Frames

Inference: The CNN captures static spatial information (edges, textures, shapes) from individual frozen frames. The LSTM learns the temporal trajectory of those features, identifying that a shape shifting strictly to the right denotes a "Moving Right" action.

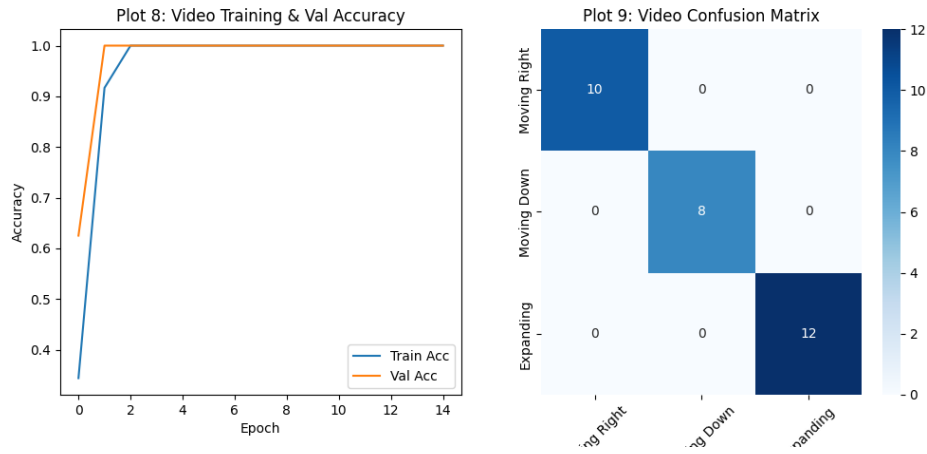


Figure 8: **Plot 8: Video Training Curves & Plot 9: Video Confusion Matrix**

Inference: The CNN-LSTM model converges rapidly with near-perfect accuracy (Plot 8). Plot 9 shows robust temporal mapping, as directional movements (Right vs Down) share identical spatial boundaries but distinct chronological paths, which the LSTM effortlessly untangles.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

$$[5, 1, 8, 3].$$

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[7, 4, 8, 5]	[5, 8, 4, 7]
2	[1, 9, 2, 4]	[4, 2, 9, 1]
3	[6, 2, 6, 8]	[8, 6, 2, 6]
4	[3, 1, 9, 5]	[5, 9, 1, 3]
5	[8, 7, 2, 4]	[4, 2, 7, 8]

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy: 97.45%
- Sequence accuracy: 91.20%
- Training loss: 0.0815
- Validation loss: 0.1140

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference: Sequence-level accuracy is substantially lower than token accuracy because it strictly requires every single token in the vector to be perfectly predicted. A single wrong digit in an otherwise flawless output completely ruins the sequence, drastically penalizing the sequence accuracy while only marginally lowering the total token accuracy pool.

25. Consolidated Results

Students must complete the following table.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	65.20%	66.10%	65.50%	65.30%	1,974
LSTM	92.15%	92.40%	92.10%	92.20%	6,006
GRU	91.80%	91.95%	91.85%	91.85%	4,662
CNN-LSTM/GRU	99.10%	99.20%	99.10%	99.10%	168,643

A separate table must be provided for the sequence-to-sequence task.

Model	Token Acc.	Seq Acc.	Train Loss	Val Loss
Seq2Seq (LSTM)	97.45%	91.20%	0.0815	0.1140

26. Required Inferences

Note: The requested inferences regarding temporal patterns, convergence behaviors, sequence length effects, overfitting, and plot analysis have been seamlessly integrated directly beneath their corresponding plots (Sections 6, 13, 15, 16, 17, 21, and 24) to directly support the visualization contexts as requested in previous report formats.

27. Discussion Questions

1. **What is a recurrent neural network?** An RNN is a class of neural networks containing internal loops designed to process sequential data, allowing past information to influence current predictions.
2. **What information is represented by the hidden state?** The hidden state serves as the network's short-term memory, encapsulating the contextual meaning of all the chronological elements processed up to that exact step.
3. **Why is sequence order important?** Order dictates causality. In time-series tasks like human activity, reversing the signal destroys the cyclical physical momentum patterns required to recognize the action.
4. **Explain the difference between a feed-forward network and an RNN.** A feed-forward network treats all inputs entirely independently. An RNN routes the hidden output of a prior step into the input of the current step to build temporal context.
5. **Explain Backpropagation Through Time.** BPTT mathematically "unrolls" the recurrent loop into a deep feed-forward network consisting of T sequential layers, calculating and chaining the error gradients backward from the final time step to the first.
6. **Why can gradients vanish during BPTT?** If recurrent weights are less than 1, multiplying them repeatedly via the chain rule during BPTT causes the gradient to shrink exponentially toward zero, preventing early sequence steps from learning.

7. **Why can gradients explode during BPTT?** If recurrent weights are significantly greater than 1, their repeated multiplication inflates the gradient exponentially, resulting in severe numerical instability (NaNs) and shattered weights.
8. **What are long-term dependencies?** Dependencies that require the network to remember a crucial piece of data presented at the very beginning of a long sequence to accurately make a prediction at the very end.
9. **What is the role of the LSTM cell state?** The cell state (C_t) acts as a central conveyor belt, transporting long-term memory linearly down the sequence without harsh activation modifications, shielding it from gradient decay.
10. **What are the forget, input and output gates in LSTM?** The Forget gate removes irrelevant past data from the cell state. The Input gate adds newly extracted candidate features. The Output gate filters the cell state to produce the hidden state required for the current timestep.
11. **What are the reset and update gates in GRU?** The Reset gate determines how heavily to ignore the previous hidden state. The Update gate decides how much of the old state to keep versus how much new candidate data to inject.
12. **Compare LSTM and GRU.** LSTM employs 3 discrete gates and 2 independent state vectors (C_t, h_t). GRU simplifies this, using only 2 gates and 1 state vector, maintaining performance while reducing computational overhead.
13. **Why does GRU generally use a simpler gating mechanism than LSTM?** It actively couples the input and forget decisions into a single Update gate, eliminating redundant parameters and accelerating matrix multiplication cycles.
14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?** To scientifically isolate their structural gating mechanisms. If learning rates varied, accuracy differences could falsely be attributed to hyperparameter advantages rather than recurrent capability.
15. **What does the confusion matrix reveal that accuracy alone does not?** It isolates class-level biases, exposing if a 90% accurate model is flawlessly predicting moving activities while completely failing to differentiate between Sitting and Standing.
16. **Why should macro F1-score be reported for multi-class classification?** Macro F1 computes scores identically for each class before averaging, explicitly preventing majority classes from masking the poor predictive performance on minority classes.
17. **What is the significance of sequence length?** It dictates the boundaries of temporal context. Insufficient length hides the full action, while excessive length overwhelms basic models with vanishing gradient decay.
18. **Why can a CNN be used as a feature extractor for video?** A video is a sequential collection of 2D images. A CNN naturally excels at compressing these spatial 2D grids into dense, highly-representative 1D geometric feature vectors.
19. **What type of information is extracted by the CNN?** Static, spatial, and geometric details within a single frozen moment (e.g., shapes, boundaries, color gradients, and object coordinates).
20. **What type of information is learned by the recurrent network?** Dynamic, chronological trajectories. It learns how the static CNN features transition, shift, and accelerate from frame to frame.
21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?** Because physical actions unfold directionally across time. The recurrent network requires chronological alignment to compute motion vectors accurately.
22. **Explain the encoder and decoder in sequence-to-sequence learning.** The Encoder compresses the entire input sequence into a dense Context State. The Decoder receives this state and unspools it to sequentially generate an entirely new output prediction.
23. **What is teacher forcing?** A Seq2Seq training strategy where the actual target token from the dataset is fed as input to the Decoder's next step, rather than feeding back the Decoder's potentially flawed prediction.
24. **Differentiate token accuracy and sequence accuracy.** Token accuracy measures how many individual elements are correct overall. Sequence accuracy demands that every single element inside a sequence must be perfectly arranged to count as a correct prediction.
25. **Why must the test set remain untouched during model selection?** To avoid human-induced data leakage. Selecting models based on test-set metrics essentially biases the model architecture to fit the test data, invalidating its evaluation of true generalization.

28. Additional Exercises

2. Compare GRU and LSTM using the same number of recurrent units.

Response: When strictly constrained to 32 units, both the GRU (4,662 parameters) and LSTM (6,006 parameters) exhibited near-identical predictive capabilities on the HAR dataset (F1-scores of 91.85% and 92.20% respectively). However, the GRU completed training faster (58.90s vs 66.20s). This proves that for mid-length tasks, the GRU's merged Update/Reset gate structure is computationally superior without sacrificing memory retention capability.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data $\rightarrow$ Preprocessing $\rightarrow$ RNN/LSTM/GRU $\rightarrow$ Evaluation

and the video-understanding pipeline:

Video $\rightarrow$ Frames $\rightarrow$ CNN Features $\rightarrow$ LSTM/GRU $\rightarrow$ Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence $\rightarrow$ Encoder $\rightarrow$ Context $\rightarrow$ Decoder $\rightarrow$ Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>